

Lab-6 学号排序实验报告

李熙正 2024302181141

一、实验目的

本实验完成基于 RISC-V 指令集的学号数字排序程序，并分别在单周期 CPU 和流水线 CPU 上运行。实验过程包括仿真验证、Vivado 工程构建、指令存储器初始化以及开发板下载测试。排序程序将 8 位学号按 BCD 形式存储，每 4 bit 表示一个十进制数字，最终将各位数字按升序排列。

二、实验环境

- 开发板：Nexys A7 / Nexys4 DDR, Artix-7 FPGA
- 开发工具：Vivado
- 仿真工具：Icarus Verilog
- 单周期 CPU 源码目录：source-sc
- 流水线 CPU 源码目录：source-pl

三、实验程序说明

排序程序首先将学号按照 BCD 编码拼接成 32 位数据。例如默认测试学号为：

```
1 | 54873530 -> 0x54873530
```

修改后的个人测试学号为：

```
1 | 13572468 -> 0x13572468
```

程序将原始学号写入数据存储器地址 0x180，将排序后的结果写入地址 0x184。排序采用选择排序思想，每一轮从未处理的数字中选择最大值并交换到对应位置。由于从低位到高位处理，最终得到升序排列结果。

个人学号构造代码如下图所示：

```

78 #####
79     addi    x2, x0, 0x13          # t
80     slli    x2, x2, 8
81     addi    x2, x2, 0x57
82     slli    x2, x2, 16
83     addi    x3, x0, 0x24
84     slli    x3, x3, 8
85     addi    x3, x3, 0x68
86     add     x2, x2, x3            # s
87     sw      x2, 0x180(x0)        # s
88     addi    x11, x0, 8           # t
89     lw      x15, 0x180(x0)       # x
90     add     x2, x0, x0           # t
91     addi    x4, x0, 0x0f         # r
92 loop1:

```

四、单周期 CPU 实现

单周期 CPU 的特点是每条指令在一个时钟周期内完成。当前工程中的 `SCCPU` 连接了控制器、寄存器堆、立即数扩展、ALU、NPC 和数据存储器等模块。每条指令在一个周期内完成取指、译码、执行、访存和写回等操作。

单周期版本主要模块包括：

- `SCCPU.v`：单周期 CPU 顶层
- `ctr1.v`：控制信号生成
- `alu.v`：算术逻辑运算
- `EXT.v`：立即数扩展
- `NPC.v`：下一条 PC 计算
- `RF.v`：寄存器堆
- `dm.v`：数据存储器

默认程序仿真结果如下，说明基础环境和默认学号排序程序可以正常运行：

```
PS D:\lxz_whu\计算机设计\ComputerDesignLabs\lab-6\source-sc> .\build.bat
[INFO] Building sccpu_sim.out ...
[INFO] Running simulation ...
VCD info: dumpfile sccpu_sid_sort.vcd opened for output.
[RESULT] original_sid=54873530
[RESULT] sorted_sid=03345578
[PASS] lab-6 sid sorting simulation passed.
sccomp_tb.v:68: $finish called at 5020 (1ns)
[OK] Waveform generated: sccpu_sid_sort.vcd
[OK] Done.
```

个人学号修改后再次运行 `.\build.bat`，得到：

```
1 | original_sid=13572468
2 | sorted_sid=12345678
```

```
PS D:\lxz_whu\计算机设计\ComputerDesignLabs\lab-6\source-sc> .\build.bat
[INFO] Building sccpu_sim.out ...
[INFO] Running simulation ...
VCD info: dumpfile sccpu_sid_sort.vcd opened for output.
[RESULT] original_sid=13572468
[RESULT] sorted_sid=12345678
[PASS] lab-6 sid sorting simulation passed.
sccomp_tb.v:68: $finish called at 10020 (1ns)
[OK] Waveform generated: sccpu_sid_sort.vcd
[OK] Done.
```

五、流水线 CPU 实现

流水线 CPU 将指令执行过程划分为多个阶段，使多条指令可以重叠执行。实验中的流水线可划分为：

```
1 | IF 取指
2 | ID 译码
3 | EX 执行
4 | MEM 访存
5 | WB 写回
```

与单周期 CPU 相比，流水线 CPU 外部运行结果一致，但内部执行方式不同。单周期 CPU 是一条指令完整执行完后再执行下一条；流水线 CPU 则让多条指令同时位于不同阶段，从而提高指令吞吐率。

流水线版本需要额外处理冒险问题：

- 数据冒险：后一条指令使用前一条指令尚未写回的寄存器结果。
- 访存冒险：`lw` 后紧跟使用加载结果时，需要暂停或前递。
- 控制冒险：`beq`、`bne`、`jal`、`jalr` 改变 PC 后，需要冲刷已经错误取出的指令。

在流水线实现中，需要补齐与单周期版本相同的指令支持，并对跳转、分支和数据相关进行处理。流水线版本最终应与单周期版本得到相同排序结果：

```
1 | mem[0x180] = 0x13572468
2 | mem[0x184] = 0x12345678
```

六、新增指令说明

类型	指令	作用
逻辑运算	and、andi、or、ori、xori	提取数字、构造掩码、清零或合并数字
比较	slt	比较两个 BCD 数字大小
移位	sll、slli、srl	按 4 bit 移动 BCD 数字
分支	beq、bne	实现循环和条件判断
跳转	jal、jalr	调用并返回 swap 子过程
高位立即数	lui	构造外设地址高 20 位

其中 `slt` 用于判断当前最大值是否小于新取出的数字，`sll` 和 `srl` 用于将对应 BCD 位移动到低 4 位或放回目标位置，`xori -1` 用于按位取反形成清除掩码。

七、jalr 返回 swap 子过程的原理

排序程序中使用 `swap` 子过程完成两个 BCD 数字位置的交换。调用时使用：

```
1 | jal x1, swap
```

`jal` 指令完成两件事：

- 1. 将下一条指令地址 `PC + 4` 写入 `x1`。
- 2. 将 PC 跳转到 `swap` 标签地址。

因此 `x1` 相当于返回地址寄存器。`swap` 子过程结束时执行：

```
1 | jalr x0, x1, 0
```

`jalr` 将下一条 PC 设置为：

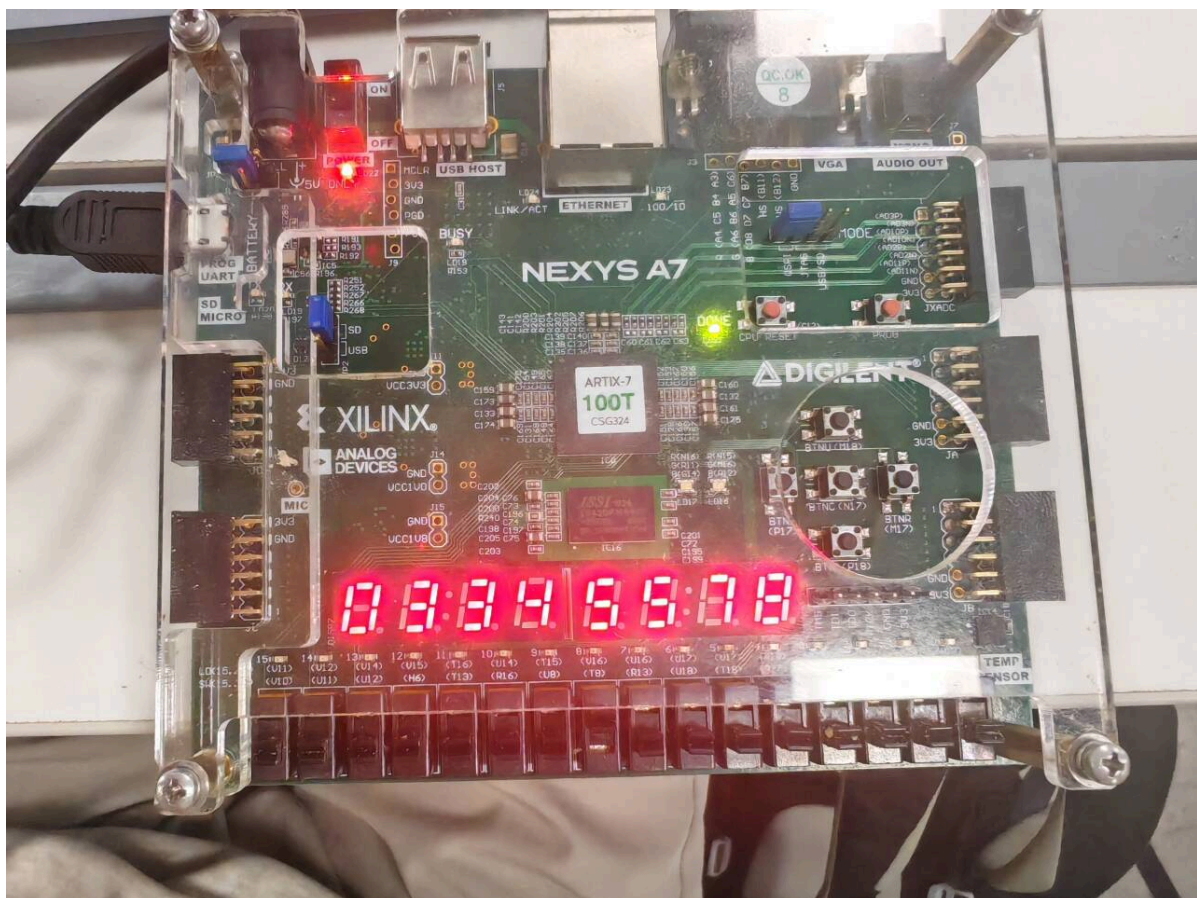
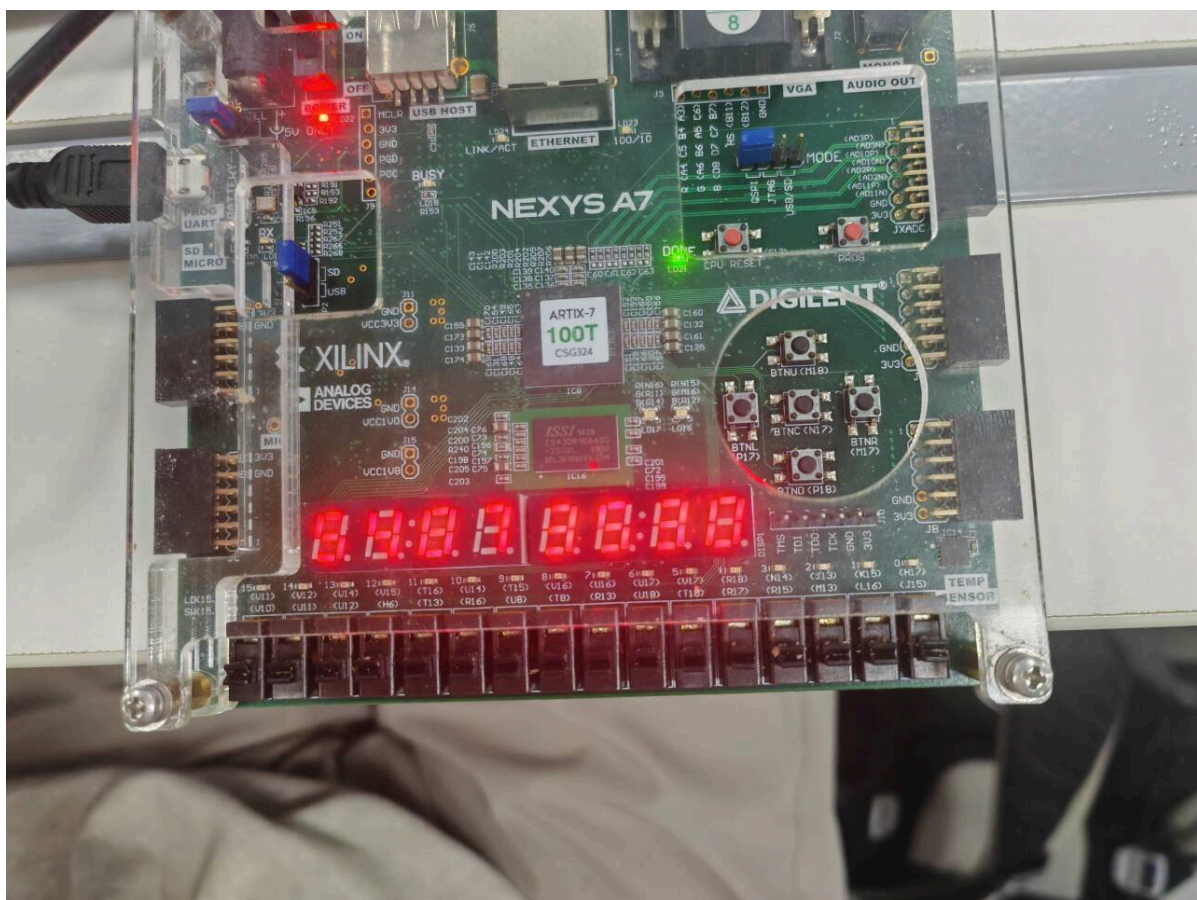
```
1 | (x1 + 0) & 0xfffffffffe
```

由于目标寄存器是 `x0`，返回时不会保存新的链接地址。这样 CPU 就会回到 `jal x1, swap` 后面的指令继续执行。该机制相当于函数调用中的“保存返回地址并返回”。

八、Vivado 上板实现

上板时使用 Vivado 创建工程，并将 CPU、数据存储器和数码管显示模块和 `imem` 指令 ROM 连接到顶层。指令 ROM 通过 `.coe` 文件初始化。需要注意的是，仿真使用的 `.dat` 文件不会自动生成 Vivado 所需的 `.coe` 文件，因此上板时需要使用与当前学号一致的 FPGA 版 `.coe` 文件。

开发板显示结果如下图所示。拨码开关用于选择显示原始学号或排序后学号：54873530->03345578



在开发板上，单周期版本和流水线版本的外部显示结果应保持一致，即都能显示原始学号和排序后的学号。两者的区别主要体现在 CPU 内部时序：单周期 CPU 每条指令一个周期完成，流水线 CPU 多条指令重叠执行，需要额外处理冒险。

九、实验结论

本实验完成了学号排序程序的仿真和开发板运行。单周期 CPU 能够正确执行排序程序，并将原始学号和排序后结果写入指定数据存储器地址。流水线 CPU 在补齐指令支持和处理数据冒险、控制冒险后，也可以运行相同的排序程序并得到一致结果。

实验中进一步理解了 RISC-V 指令在 CPU 数据通路中的执行方式，掌握了 `jal/jalr` 实现子过程调用与返回的机制，也明确了仿真 `.dat` 文件和 Vivado ROM 初始化 `.coe` 文件之间的区别。